# Zero to Hero Study Handbook: local-ai-chat-studio

## Module 1: Foundations & Architecture

### What this project does

`local-ai-chat-studio` is a local-first Streamlit application that provides a ChatGPT-style interface on top of: - Local Ollama models (default path). - Optional BYOK cloud providers (`openai`, `anthropic`, `openrouter`, `xai`, `gemini`). - Long-term memory and semantic retrieval over uploaded files and past chats.

Main use cases supported by code: - Multi-turn assistant chat with streaming output (`app.py`, `src/jobs.py`). - File-assisted chat over PDFs, Office docs, spreadsheets, text/code, and images (`src/files.py`, `src/jobs.py`). - Cross-chat semantic recall and long-term memory injection (`src/rag.py`, `src/memory.py`, `src/orchestrator.py`). - Provider key management and dynamic model discovery (`pages/3_Providers.py`, `src/providers.py`, `src/catalog.py`). - Side-by-side model comparison (`pages/4_Compare.py`, `src/jobs.py`).

### Core paradigms and patterns used here

Definition first, then where it appears:

1. Local-first architecture. The app stores chats/vectors locally by default (`data/app.db`, `data/chroma`) and talks to local Ollama unless you explicitly configure cloud endpoints.
2. Event-driven UI with reruns. Streamlit reruns scripts on interaction; this app uses `st.session_state`, `st.fragment(run_every=...)`, and callbacks to keep state consistent while rendering live updates (`app.py`, `pages/*.py`).
3. Background worker pattern. Long-running generation runs in daemon threads via a process-global `Job` registry (`src/jobs.py`) so the UI thread stays responsive.
4. Retrieval-augmented generation (RAG) pipeline. Uploaded docs and chat history are embedded and queried through ChromaDB (`src/rag.py`), then injected into system/context messages (`src/orchestrator.py`).
5. Functional module style with typed data carriers. Most logic is organized as module-level functions, with small dataclasses for structured data (`ModelInfo`, `ApiModel`, `SelectedModel`, `Attachment`, `Job`).

### Architecture and component interaction

Primary runtime components: - UI and orchestration shell: `app.py`. - Stateful pages: `pages/1_Memory.py`, `pages/2_Settings.py`, `pages/3_Providers.py`, `pages/4_Compare.py`. - Persistence: SQLite (`src/chat_store.py`). -

Retrieval/vector index: Chroma (`src/rag.py`). - Generation workers: `src/jobs.py`. - Prompt/context assembly: `src/orchestrator.py`. - Provider adapters and key vault: `src/providers.py`. - Ollama client and model capabilities: `src/ollama_client.py`. - Config and storage paths: `src/config.py`.

ASCII main-flow diagram:

```
User (Streamlit UI)
  -> app.py (send_user_message)
    -> chat_store.add_message(role='user')
    -> jobs.start(...)
      -> background thread _run
        -> _process_attachments
           -> files.parse_upload / files.chunk_text
           -> rag.index_doc_chunks (for large docs)
        -> orchestrator.build_messages
           -> personalization.get_profile
           -> memory.relevant_memories
           -> rag.search_history / rag.search_docs
        -> _stream
           -> ollama_client.stream_chat OR providers.stream_chat
        -> chat_store.add_message(role='assistant', attachments=meta)
        -> post-processing
           -> orchestrator.after_turn_indexing
           -> memory.extract_memories (periodic)
           -> personalization.rebuild_profile (periodic)

Persistent state:
  SQLite: data/app.db (conversations, messages, memories, feedback, kv, presets)
  Chroma: data/chroma (doc chunks, chat history, memories)
```

## Module 2: Repository Map

| File/Directory Path | Primary Responsibility | Key Classes/Functions | Important Configs/Variables |
|---|---|---|---|
| `app.py` | Main Chat page, sidebar navigation, live streaming UI, chat actions | `send_user_message`, `start_generation`, `regenerate_last`, `render_live_generation`, `render_health` | `MEMORY_EXTRACT_EVERY`, session keys (`selected_model`, `active_id`, `settings_*`) |
| `pages/1_Memory.py` | Memory/profile management UI | `get_profile`, `rebuild_profile`, `chat_store.update_memory`, `rag.delete_memory_vector` | Uses `st.session_state.helper_model` |

| File/Directory Path | Primary Responsibility | Key Classes/Functions | Important Configs/Variables |
|---|---|---|---|
| pages/2_Settings.py | Generation/features toggles, maintenance, import/export, wipe controls | memory.run_decay, chat_store.export_jsonl, chat_store.import_jsonl, chat_store.wipe_everything | settings_temperature, settings_system_prompt, settings_memory, settings_crosschat, settings_show_cloud |
| pages/3_Providers.py | API key entry, OpenRouter OAuth, Ollama endpoint config | providers.set_api_key, providers.test_connection, providers.openrouter_auth_url, providers.set_ollama_state | Provider env vars, OLLAMA_CLOUD_URL, OpenRouter PKCE state/key |
| pages/4_Compare.py | Side-by-side ephemeral generation across two models | jobs.run_ephemeral, render_compare, build_model_catalog | KEY_A, KEY_B, cmp_* session keys |
| src/config.py | Typed app settings and data directories | AppConfig, ensure_dirs, db_path, chroma_dir | CHAT_ env prefix, .env, data_dir, request_timeout, retrieval/memory tuning fields |
| src/chat_store.py | SQLite schema and data access layer for chats/memories/profiles/presets | init_db, create_conversation, add_message, list_memories, set_feedback, kv_get/kv_set, export_jsonl | _SCHEMA, tables: conversations, messages, memories, feedback, kv, presets |
| src/jobs.py | Background generation lifecycle and cancellation | Job, start, _run, _process_attachments, _stream, run_ephemeral | _jobs registry, _lock, MEMORY_EXTRACT_EVERY |
| src/orchestrator.py | Build final message stack (system + memory + retrieval + history) | build_messages, after_turn_indexing | BASE_SYSTEM, config.cross_chat_top_k, config.rag_top_k |
| src/rag.py | Chroma indexing/query for docs/history/memories | index_doc_chunks, search_docs, index_history_turn, search_history, search_memories | Collections: doc_chunks, chat_history, memories |

| File/Directory Path | Primary Responsibility | Key Classes/Functions | Important Configs/Variables |
|---|---|---|---|
| src/memory.py | Extract durable user facts and retrieve relevant memories | extract_memories, relevant_memories, run_decay | _EXTRACT_SYSTEM, config.memory_max_injected, dedup threshold in rag.similar_memory |
| src/personalization.py | Rolling profile rebuild based on chats + feedback | get_profile, note_conversation_end, rebuild_profile | KV keys: user_profile, convs_since_profile, config.profile_refresh_every |
| src/providers.py | Provider catalog metadata, key vault, streaming adapters, OAuth helpers | ApiModel, list_provider_models, stream_chat, openrouter_auth_url | PROVIDERS, in-memory _secrets, env fallbacks (OPENAI_API_KEY, etc.), _OLLAMA_HOST_KEY, _OLLAMA_API_KEY |
| src/ollama_client.py | Ollama model discovery, health, chat streaming, embeddings, vision fallback | ModelInfo, list_models, stream_chat, embed_texts, describe_image, ollama_alive | _client_cache, config.keep_alive, config.request_timeout |
| src/catalog.py | Unified model catalog across local/cloud providers | SelectedModel, build_model_catalog, ordered_keys, best_coding_model | Key format (ollama::<name>, <provider>::<id>), provider group ranking |
| src/files.py | Parse uploaded files and chunk text | Attachment, parse_upload, _parse_pdf, _parse_docx, _parse_excel, chunk_text | IMAGE_TYPES, DOC_TYPES, ACCEPTED_TYPES, MIME_BY_EXT |
| .streamlit/config.toml | Streamlit app theme/client/server defaults | N/A | server.maxUploadSize=100, theme colors, browser.gatherUsageStats=false |

| | | | |
|---|---|---|---|
| `pyproject.toml` | Project metadata and dependencies | N/A | Python `>=3.12`, runtime deps (`streamlit`, `ollama`, `chromadb`, provider SDKs) |

## Module 3: Core Execution Flows

### Flow A: Startup and model catalog build (main chat path)

Step-by-step: 1. `app.py` initializes page config and ensures DB schema with `chat_store.init_db()`. 2. Health gate: `ollama_alive()` must pass or app stops with an error. 3. `cached_models()` calls `list_models()` to fetch Ollama model metadata. 4. Provider model lists are fetched only for configured providers (`providers.configured_providers()`). 5. `build_model_catalog(models, show_cloud, provider_models)` creates one unified catalog of selectable models. 6. Embedding/helper models are auto-selected using: - `embedding_model(models)` for vectors/memory. - `smallest_text_model(models)` for helper tasks (titles/profile/memory extraction). 7. `seed_builtin_presets(catalog)` inserts built-in Coding Agent preset if missing.

Core data shape used by the UI (`SelectedModel` in `src/catalog.py`):

```
SelectedModel(
    key: str,          # 'ollama::<name>' or '<provider>::<id>'
    provider: str,     # 'ollama' | 'openai' | 'anthropic' | ...
    name: str,
    hint: str,
    detail: str,
    is_vision: bool,
    group: str,
    group_rank: int,
    context_length: int | None,
)
```

### Flow B: User sends a message and receives a streamed answer

Step-by-step: 1. UI receives `st.chat_input(...)` in `app.py`. 2. `send_user_message(user_text, raw_files)` runs: - Create conversation if needed (`chat_store.create_conversation`). - Persist user message immediately via `chat_store.add_message(..., role='user', attachments=att_meta)`. - Queue background generation with `start_generation(...)` -> `jobs.start(...)`. 3. `jobs.start` creates a `Job` object and daemon thread that runs `_run(...)`. 4. `_run` phase `preparing`: - `_process_attachments` parses files with `files.parse_upload`. - Small docs are injected directly;

large docs are chunked (`chunk_text`) and indexed (`rag.index_doc_chunks`). - Image handling depends on model capability: - Vision-capable target model: raw images passed forward. - Text-only target model: optional OCR/description through `describe_image` fallback. - Optional web search via `_web_search` (DuckDuckGo) and source list assembly. 5. `_run` builds model input: - `orchestrator.build_messages(...)` assembles system/profile/memory/retrieval/history. - Current user turn is appended; images added as `images` field when present. 6. `_run` phase `generating`: - `_stream(...)` dispatches to `ollama_stream` or `providers.stream_chat`. - Partial chunks update `job.text` for live UI fragment rendering. 7. Completion: - Assistant message persisted via `chat_store.add_message(role='assistant', content=answer, attachments=meta)`. - `meta` includes reference titles, optional web sources, and runtime stats (`secs`, `tokens`, `tok_s`). 8. Post-processing (best effort): - `_autotitle` on first turn. - `after_turn_indexing` for cross-chat vectors. - periodic `_maybe_extract` for memory/profile refresh.

Internal message shape expected by generation functions:

```
{"role": "user" | "assistant" | "system", "content": str, "images": [(bytes, mime)]?}
```

SQLite `messages` row as returned by `chat_store.get_messages`:

```
{
  "id": str,
  "conv_id": str,
  "role": "user" | "assistant",
  "content": str,
  "attachments_json": str | None,
  "ts": str,
  "attachments": list[dict],
}
```

`Job` runtime state (`src/jobs.py`):

```
Job(
  conv_id: str,
  model_name: str,
  status: "running" | "done" | "error" | "cancelled",
  phase: "preparing" | "generating",
  text: str,
  error: str,
  references: list[str],
  notes: list[str],
  cancelled: bool,
)
```

### Flow C: Context assembly (memory + references + documents)

`build_messages(...)` in `src/orchestrator.py` constructs the final prompt in this order: 1. Base or custom system prompt. 2. User profile from `personalization.get_profile()` when memory is enabled. 3. Long-term memories from `memory.relevant_memories(...)`. 4. Cross-chat semantic hits from `rag.search_history(...)` with configurable `min_similarity`. 5. Uploaded document retrieval snippets from `rag.search_docs(...)` when docs exist for that conversation. 6. Attachment context (parsed docs/OCR/web search text). 7. Last up to 20 history turns.

Return value shape:

```
(messages: list[dict[str, Any]], reference_titles: list[str])
```

Cross-chat/doc/memory query result shape from `rag._flatten`:

```
{"id": str, "text": str, "similarity": float, ...metadata}
```

### Flow D: Memory extraction and profile refresh loop

Memory loop (`src/memory.py`, `src/jobs.py`): 1. Trigger condition: `_maybe_extract` runs when message count is a multiple of `MEMORY_EXTRACT_EVERY` (8). 2. `memory.extract_memories` creates a transcript of recent turns and calls `ollama_client.generate` with `_EXTRACT_SYSTEM` JSON instructions. 3. Extracted facts are parsed by `_parse_json_list`, deduplicated by vector similarity (`rag.similar_memory`), then inserted into SQLite + Chroma. 4. Conversation marker `memory_extracted_at` is updated.

Profile loop (`src/personalization.py`): 1. `note_conversation_done` increments counter in `kv`. 2. When counter reaches `config.profile_refresh_every` (default 5), `rebuild_profile` regenerates bullet-profile text from recent user snippets + rated assistant responses. 3. Profile text is stored in `kv` key `user_profile`.

### Flow E: Provider and endpoint management

Provider path (`pages/3_Providers.py`, `src/providers.py`): 1. User sets key in UI -> `providers.set_api_key(provider, key)` stores it in in-memory `_secrets`. 2. If no session key exists, read-only env fallback applies via `get_api_key`. 3. `list_provider_models` fetches live model lists per provider. 4. `providers.stream_chat` adapts internal message format to provider-specific streaming APIs.

Ollama endpoint path: 1. Host/API key managed via `providers.set_ollama_config(host, api_key)`. 2. `src/ollama_client._client()` uses `(host, key)` signature to rebuild cached client. 3. All model listing, streaming, embedding, and health checks use this configured endpoint.

## Module 4: Setup & Run Guide

### 1. Prerequisites

- Python >=3.12 (from pyproject.toml).
- uv for dependency management.
- Ollama endpoint reachable (local or remote/cloud).
- At least one chat model available in Ollama.
- Optional for .doc parsing in src/files.py: antiword or soffice (LibreOffice).

### 2. Install dependencies

```
uv sync
```

Dependencies are declared in pyproject.toml, including: - UI/runtime: streamlit, loguru. - Local inference: ollama. - Retrieval: chromadb. - Providers: openai, anthropic, httpx. - File parsing: pypdf, python-docx, pandas, openpyxl, xlrd, pillow.

### 3. Run the app

```
uv run streamlit run app.py
```

Optional custom port (as documented in README):

```
uv run streamlit run app.py --server.port 8503
```

### 4. Environment variables and config

Config loading source: - src/config.py uses SettingsConfigDict(env_prefix="CHAT_", env_file=".env"). - Any AppConfig field can be overridden via CHAT_<FIELD_NAME_IN_UPPERCASE>.

Common CHAT_ keys used by runtime: - CHAT_OLLAMA_HOST - CHAT_DATA_DIR - CHAT_TEMPERATURE - CHAT_KEEP_ALIVE - CHAT_REQUEST_TIMEOUT - CHAT_SHOW_CLOUD_MODELS - CHAT_MEMORY_ENABLED - CHAT_CROSS_CHAT_REFERENCES - CHAT_CHUNK_CHARS - CHAT_CHUNK_OVERLAP_CHARS - CHAT_DOC_CONTEXT_BUDGET_CHARS - CHAT_RAG_TOP_K - CHAT_CROSS_CHAT_TOP_K - CHAT_CROSS_CHAT_MIN_SIMILARITY - CHAT_MEMORY_MAX_INJECTED - CHAT_MEMORY_DECAY_DAYS - CHAT_PROFILE_REFRESH_EVERY

Provider/API keys supported by code (src/providers.py): - OPENAI_API_KEY - ANTHROPIC_API_KEY - OPENROUTER_API_KEY - XAI_API_KEY - GEMINI_API_KEY - OLLAMA_API_KEY (for secured/hosted Ollama endpoint)

UI/runtime config files: - .streamlit/config.toml for theme, server.maxUploadSize, telemetry toggle. - .env optional for CHAT_* overrides.

### 5. Data storage locations

Derived from `config.data_dir` (default `<repo>/data`): - SQLite DB: `data/app.db` - Chroma persistent store: `data/chroma` - Uploads directory: `data/uploads`

### 6. Migrations, seeding, and initialization

There is no separate migration command. Runtime initialization handles it: - `chat_store.init_db()` creates schema on startup for all pages. - Lightweight migration attempt in `init_db`: adds `conversations.pinned` if missing. - Built-in assistant seeding in `app.py`: `seed_builtin_presets(...)` ensures the built-in coding preset exists when a catalog is available.

### 7. Typical startup sequence on a clean machine

```
# 1) install deps
uv sync

# 2) (recommended) ensure an embedding model exists for memory/RAG
ollama pull embeddinggemma

# 3) run app
uv run streamlit run app.py
```

If you use cloud-only/provider models, configure keys and endpoint from `pages/3_Providers.py` UI.

## Module 5: Study Plan & Practice Exercises

### Ordered study plan for a new learner

1. Read `README.md` to understand product goals, feature set, and runtime assumptions.
2. Read `src/config.py` and `src/chat_store.py` to understand core state, persistence schema, and data paths.
3. Read `app.py` top-to-bottom for user journey and orchestration entrypoints.
4. Read `src/jobs.py` for the non-blocking execution design and generation lifecycle.
5. Read `src/orchestrator.py`, `src/rag.py`, `src/memory.py`, `src/personalization.py` to understand context assembly and personalization loops.
6. Read `src/providers.py` and `src/ollama_client.py` for model/provider adapters and endpoint behavior.
7. Read `pages/2_Settings.py`, `pages/1_Memory.py`, `pages/3_Providers.py`, `pages/4_Compare.py` for operational controls and alternate flows.
8. Read `src/catalog.py`, `src/model_labels.py`, `src/files.py` for model UX and file preprocessing details.

**Practice exercises (with solution outlines)**

1. Trace one full chat turn from UI input to assistant persistence. Solution outline: follow `app.py st.chat_input -> send_user_message -> jobs.start -> jobs._run -> _stream -> chat_store.add_message(role='assistant')`.
2. Identify where and how message metadata is stored for references, sources, and timing. Solution outline: inspect `jobs._run meta` list construction and how it is saved in `chat_store.add_message(..., attachments=meta)`; inspect rendering in `app.py` where `kind` is parsed (`reference`, `source`, `meta`, `error`).
3. Explain the exact condition for memory extraction and profile rebuild. Solution outline: extraction trigger is in `jobs._maybe_extract` when `message_count % MEMORY_EXTRACT_EVERY == 0` and not already extracted; profile rebuild is gated by `personalization.note_conversation_done()` counter vs `config.profile_refresh_every`.
4. Show how cloud provider models appear in the same dropdown as local models. Solution outline: provider keys -> `providers.configured_providers()` -> per-provider `cached_provider_models` -> `catalog.build_model_catalog` merges into `SelectedModel` dict -> `ordered_keys` drives selectbox in `app.py`.
5. Determine the fallback behavior when the selected model cannot process images. Solution outline: `jobs._process_attachments` checks `is_vision`; if false and `vision_fallback` exists, calls `describe_image` and injects extracted text context; otherwise adds a note that no vision model is available.
6. Find where cross-chat semantic search is used and how current conversation is excluded. Solution outline: `orchestrator.build_messages` calls `rag.search_history`; that function queries Chroma with `where={"conv_id": {"$ne": exclude_conv}}` and filters by `min_similarity`.
7. Confirm how secret handling avoids disk persistence. Solution outline: `src/providers.py` stores keys in process memory `_secrets` with lock; functions use env vars as read-only fallback; comments and page text explicitly state session-only behavior; no DB/file write path for keys.
8. Explain what is deleted by each cleanup action. Solution outline: read `pages/2_Settings.py` and `chat_store.py/rag.py`: `Clear all chats` deletes conversations/messages/feedback + chat vectors only; `Panic wipe` deletes conversations, memories, kv, presets, all vector collections, and in-memory secrets.
9. Reconstruct database schema and identify the join used for feedback retrieval. Solution outline: inspect `_SCHEMA` in `chat_store.py`; `get_feedback` joins `feedback` and `messages` on `message_id` filtered by `conv_id`.

10. Compare persistent vs ephemeral generation paths. Solution outline: persistent path uses `jobs.start/_run` and stores assistant turn; ephemeral compare path uses `jobs.run_ephemeral` with keys (`compare::a`, `compare::b`) and never writes to SQLite.

**Self-check checklist**

- Can you explain how `app.py` delegates slow work to `src/jobs.py` without blocking UI reruns?
- Can you describe the message and metadata shapes passed between `app.py`, `src/jobs.py`, and `src/chat_store.py`?
- Can you explain when `src/orchestrator.py` injects profile, memories, cross-chat hits, and doc excerpts?
- Can you list which features require an embedding model and where this dependency is checked?
- Can you explain provider key flow (session memory vs env fallback) and where secrets are stored?
- Can you trace how uploaded files are parsed, chunked, indexed, and later retrieved for context?
- Can you explain the difference between `Clear all chats` and `Panic wipe` using actual functions?
- Can you identify where model catalogs are built and how model keys are normalized across old/new conversations?